

Checklist – Aula API RG

1 Model inicial

- ☐ Criar classe **RgModel** com 3 campos iniciais
- ☐ Adicionar **[Table("RG")]** para nome da tabela no banco
- ☐ `UrlFoto` → `[Required] string`
- 💡 Usar exemplos de fotos do link: <https://randomuser.me/api/portraits/>
- ☐ `Nome` → `[Required] string`
- ☐ `DataEmissao` → `[Required] DateTime` ou `DateOnly`
- ☐ Explicar cada campo e Data Annotations

2 Scaffold

- ☐ Perguntar se querem criar na mão ou usar Visual Studio
- ☐ Executar Scaffold para gerar DbContext, DbSet e Controller CRUD
- ☐ Revisar connection string no **appsettings.json**
- ☐ Conferir **Program.cs** para registrar DbContext
- 💡 Alterar DbSet do RgModel para **RgDbSet** para evitar confusão com a classe
- 💡 Mostrar Controller e DbContext juntos, explicando como o controller usa o context

3 Primeira Migration & Teste

- ☐ Criar primeira migration (`Add-Migration Inicial`)
- ☐ Aplicar no banco (`Update-Database`)
- ☐ Popular banco via POST único
- ☐ Testar GET para verificar dados

4 Manipulação de listas

💡 Pergunta inicial: "Se eu pedir para vocês inserir 30 registros de RG como os que fiz aqui, o que vocês acham que deveriam fazer?"

- ☐ Alterar endpoint POST para receber **ICollection<RgModel>**
- ☐ Popular banco com vários RGs rapidamente
- ☐ Inserir um item intencionalmente inválido
- ☐ Mostrar retorno 400 Bad Request
- ☐ Conferir que nenhum registro foi salvo

💡 Explicar atomicidade: Se algum registro falhar, **nenhum é salvo**, como uma transação PIX

💡 Demonstrar passo a passo que a persistência só ocorre após chamar **SaveChangesAsync()**

Exemplo de loop com lista e SaveChangesAsync:

```
foreach (var item in listaRgs)
{
    _context.RgDbSet.Add(item);
}
await _context.SaveChangesAsync();
```

5 Reversão para objeto único

- ☐ Voltar endpoint para um RG por vez
- ☐ Discutir boas práticas de API

6 CORS

- ☐ Explicar problema de origem cruzada
- ☐ Configurar `app.UseCors(politica => politica.AllowAnyOrigin())`
- ☐ Testar no Postman
- ☐ Reforçar que só deve ser usado em desenvolvimento

7 Status HTTP

- ☐ Mostrar códigos HTTP: 200–299 sucesso, 400 Bad Request
- ☐ Demonstrar no Postman que 400 vem com detalhes do campo falho

8 Exercício guiado

- ☐ Pedir que eles olhem um RG na internet
- ☐ Criem novos campos no Model usando Data Annotations
- ☐ Gerar nova migration e aplicar (`Add-Migration + Update-Database`)
- ☐ Popular banco e testar endpoints

9 Exercício final – Projeto próprio

- ☐ Criar um novo projeto API do zero
- ☐ Criar Model para documento ou carteirinha de estudante
- ☐ Usar Data Annotations em campos obrigatórios

💡 Para campo de foto, usar URLs de exemplo do link: <https://randomuser.me/api/portraits/>

- ☐ Alterar o sufixo do DbSet de Model para DbSet
- ☐ Criar Controller CRUD (POST e GET mínimo)
- ☐ Gerar migration e atualizar banco
- ☐ Testar endpoints no Postman (validar 200/400)
- ☐ Opcional: adicionar campos extras, gerar nova migration, observar warnings